

CSC 345 - Project #4: Report

Checkpoint Completed: Checkpoint 3

Extra Credit Attempted: Extra 1 and Extra 2

Checkpoint 1:

Requirement 1: Server displays up-to-date connected client list whenever someone connects or disconnects.

Server:

```
sofiafhall@Leandra:~/project4$ ./main_server
Server started on port 8080
Connected: 127.0.0.1
Connected Clients:
  unknown (127.0.0.1)
Connected Clients:
No clients connected
```

Figure 1. Requirement 1 Server shows clients connecting and disconnecting

Client:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1
Trying connecting to 127.0.0.1
Connected to server
```

Figure 2. Requirement 1 Client connects to server

Requirement 2: Server accepts multiple clients and broadcasts messages to all connected clients.

Client 1:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1
Trying connecting to 127.0.0.1
Connected to server
Terminal 2: Hello! How are you?
Terminal 3: I'm good. How are you doing?
```

Figure 3. Client 1 and Client 2 broadcasts a conversation between each other

Client 2:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1
Trying connecting to 127.0.0.1
Connected to server
Terminal 2: Hello! How are you?
Terminal 3: I'm good. How are you doing?
```

Figure 4. Client 1 and Client 2 broadcasts a conversation between each other

Requirement 3: Client specifies a username when connecting to server and broadcasts usernames.

Server:

```
sofiafhall@Leandra:~/project4$ ./main_server
Server started on port 8080
Connected: 127.0.0.1
Connected Clients:
  Sofia (127.0.0.1)
Connected: 127.0.0.1
Connected Clients:
  Sofia (127.0.0.1)
  Nick (127.0.0.1)
```

Figure 5. Server shows usernames and IP addresses of all clients

Client 1:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1
Trying connecting to 127.0.0.1
Connected to server
Type your user name: Sofia
Hey Nick, how are you doing?

[Nick (127.0.0.1)] I'm good Sofia, thanks for asking. How are you?
```

Figure 6. Client 1 and Client have a conversation with usernames and IP addresses shown

Client 2:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1
Trying connecting to 127.0.0.1
Connected to server
Type your user name: Nick

[Sofia (127.0.0.1)] Hey Nick, how are you doing?
I'm good Sofia, thanks for asking. How are you?
```

Figure 7. Client 1 and Client have a conversation with usernames and IP addresses shown

Requirement 4: Join/leave notifications sent to all clients when someone connects or disconnects.

Requirement 5: Random unique colors per user within each client.

Server:

```
sofiafhall@Leandra:~/project4$ ./main_server
Server started on port 8080
Connected: 127.0.0.1
Connected: 127.0.0.1
Connected: 127.0.0.1
Connected Clients:
  Sofia (127.0.0.1)
  unknown (127.0.0.1)
  unknown (127.0.0.1)
Connected Clients:
  Sofia (127.0.0.1)
  Nick (127.0.0.1)
  unknown (127.0.0.1)
Connected Clients:
  Sofia (127.0.0.1)
  Nick (127.0.0.1)
  John (127.0.0.1)
Connected Clients:
  Nick (127.0.0.1)
  John (127.0.0.1)
Connected Clients:
  John (127.0.0.1)
Connected Clients:
No clients connected
```

Figure 8. Server shows usernames and IP addresses of all clients

Client 1:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1
Trying connecting to 127.0.0.1
Connected to server
Type your user name: Sofia

Nick (127.0.0.1) joined the chat room!

John (127.0.0.1) joined the chat room!

[John (127.0.0.1)] Hi guys! What should we talk about?

[Nick (127.0.0.1)] Computer science?

Great idea!
Nevermind, gotta go!
^C
```

Figure 9. Conversation between 3 clients show join/leave notifications and different colors for different members.

Client 2:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1
Trying connecting to 127.0.0.1
Connected to server
Type your user name: Nick

Sofia (127.0.0.1) joined the chat room!

John (127.0.0.1) joined the chat room!

[John (127.0.0.1)] Hi guys! What should we talk about?
Computer science?
[Sofia (127.0.0.1)] Great idea!

[Sofia (127.0.0.1)] Nevermind, gotta go!

Sofia (127.0.0.1) left the room!

Bye!
^C
```

Figure 10. Conversation between 3 clients show join/leave notifications and different colors for different members.

Client 3:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1
Trying connecting to 127.0.0.1
Connected to server
Type your user name: John

Sofia (127.0.0.1) joined the chat room!
Nick (127.0.0.1) joined the chat room!

Hi guys! What should we talk about?
[Nick (127.0.0.1)] Computer science?

[Sofia (127.0.0.1)] Great idea!

[Sofia (127.0.0.1)] Nevermind, gotta go!

Sofia (127.0.0.1) left the room!

[Nick (127.0.0.1)] Bye!

Nick (127.0.0.1) left the room!

^C
```

Figure 11. Conversation between 3 clients show join/leave notifications and different colors for different members.

Checkpoint 2:

Requirement 6: Multiple chat rooms simultaneously

Server:

```
sofiafhall@Leandra:~/project4$ ./main_server
Server started on port 8080
Connected: 127.0.0.1
Created room 1
Connected Clients:
[Room 1] Sof (127.0.0.1)
Connected: 127.0.0.1
Joined room 1
Connected Clients:
[Room 1] Sof (127.0.0.1)
[Room 1] Nick (127.0.0.1)
Connected: 127.0.0.1
Connected: 127.0.0.1
Connected: 127.0.0.1
Connected: 127.0.0.1
Created room 2
Connected Clients:
[Room 1] Sof (127.0.0.1)
[Room 1] Nick (127.0.0.1)
[Room 2] John (127.0.0.1)
Connected Clients:
[Room 1] Nick (127.0.0.1)
[Room 2] John (127.0.0.1)
```

Figure 12. Server shows various rooms clients

Client 1:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1 new
Trying connecting to 127.0.0.1
Connected to 127.0.0.1 with new room number 1
Type your user name: Sof

Nick (127.0.0.1) joined the chat room!

Hey Nick!
^C
```

Figure 13. Client 1 creates new room (room 1)

Client 2:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1 1
Trying connecting to 127.0.0.1
Connected to room 1
Type your user name: Nick

[SoF (127.0.0.1)] Hey Nick!

Sof (127.0.0.1) left the room!
```

Figure 14. Client 2 joins room 1 and chats

Client 3:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1 99
Trying connecting to 127.0.0.1
Room does not exist.
```

Figure 15. Client 3 attempts to join room 99 and room does not exist (past size)

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1 new
Trying connecting to 127.0.0.1
Connected to 127.0.0.1 with new room number 2
Type your user name: John
```

Figure 16. Client 3 creates new room (room 2)

Checkpoint 3:

Requirement 7: Shows a room list, lets users pick rooms or type new to create a new one. If no rooms exist, it automatically creates one.

Server:

```
sofiafhall@Leandra:~/project4$ ./main_server
Server started on port 8080
Connected: 127.0.0.1
Auto created room 1
Connected Clients:
[Room 1] Sofia (127.0.0.1)
Connected: 127.0.0.1
Joined room 1
Connected Clients:
[Room 1] Sofia (127.0.0.1)
[Room 1] Nick (127.0.0.1)
Connected: 127.0.0.1
Created room 2
Connected Clients:
[Room 1] Sofia (127.0.0.1)
[Room 1] Nick (127.0.0.1)
[Room 2] John (127.0.0.1)
Connected Clients:
[Room 1] Sofia (127.0.0.1)
[Room 2] John (127.0.0.1)
Room 2 is now empty and closed.
Connected Clients:
[Room 1] Sofia (127.0.0.1)
Room 1 is now empty and closed.
Connected Clients:
No clients connected
```

Figure 17. Server shows when rooms are created, joined, and closed.

Client 1:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1
Trying connecting to 127.0.0.1
Connected to 127.0.0.1 with new room number 1
Type your user name: Sofia

Nick (127.0.0.1) joined the chat room!

[Nick (127.0.0.1)] Hey sofia!

Hi nick. What's up?

[Nick (127.0.0.1)] Nothing much. See ya later.

Nick (127.0.0.1) left the room!

sofiafhall@Leandra:~/project4$
```

Figure 18. No arguments shows no existing rooms so automatically creates and joins Client 1 into room number 1

Client 2:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1
Trying connecting to 127.0.0.1
Server says the following options are available:
Room 1: 1 person
Choose the room number or type [new] to create a new room: 1
Connected to room 1
Type your user name: Nick
Hey sofia!

[Sofia (127.0.0.1)] Hi nick. What's up?

Nothing much. See ya later.

sofiafhall@Leandra:~/project4$
```

Figure 19. No argument after a room exists shows a list of rooms and number of people in them

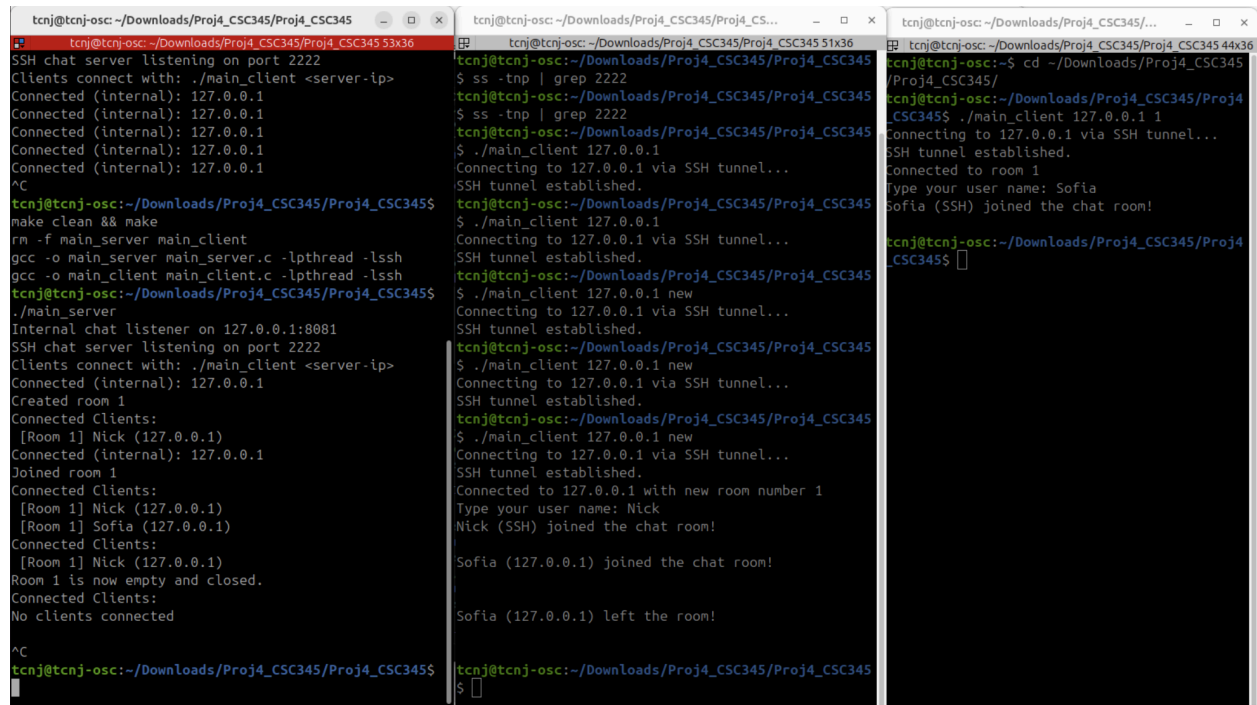
Client 3:

```
sofiafhall@Leandra:~/project4$ ./main_client 127.0.0.1
Trying connecting to 127.0.0.1
Server says the following options are available:
Room 1: 2 people
Choose the room number or type [new] to create a new room: new
Connected to room 2
Type your user name: John
Where's everyone at?

sofiafhall@Leandra:~/project4$
```

Figure 20. Client 3 joins a room by creating a new one

Extra Credit 1:



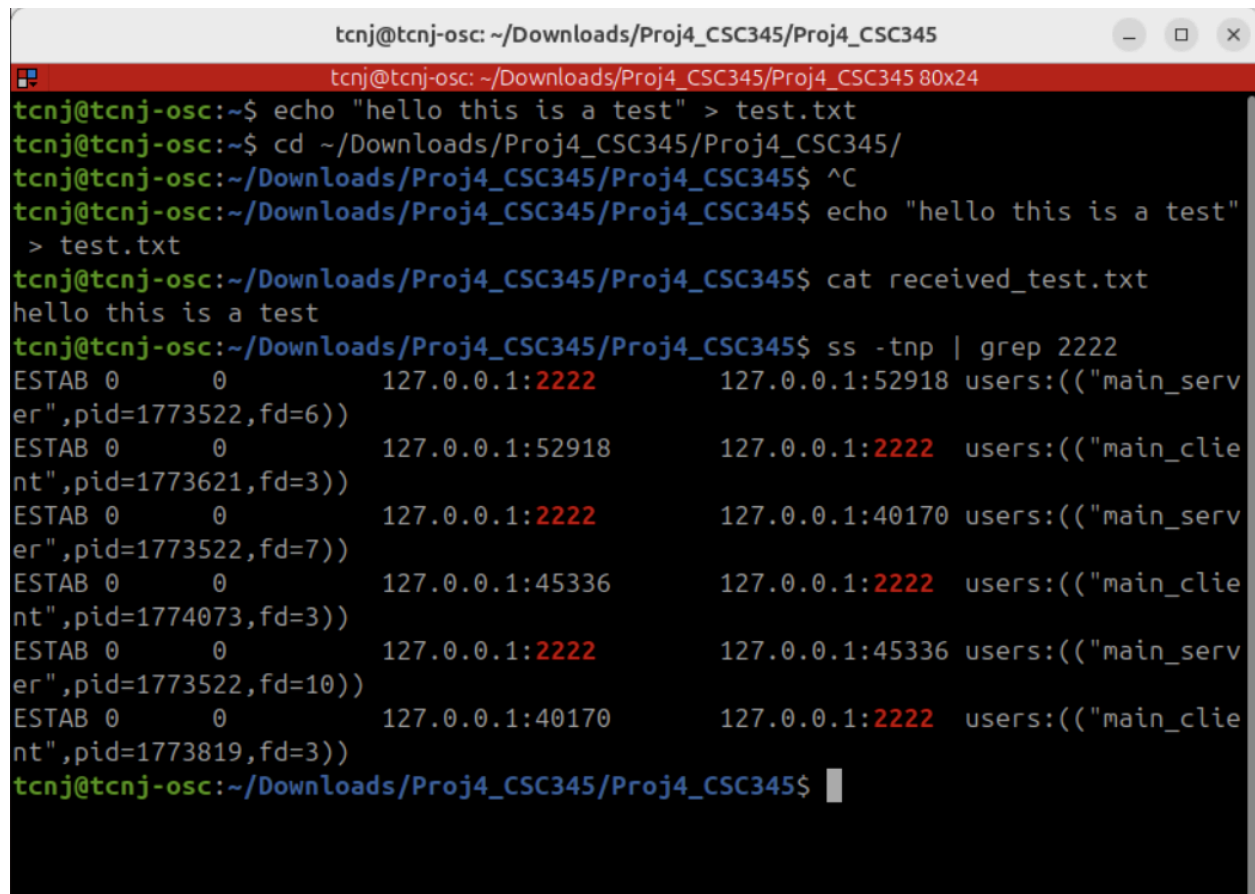
```
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
SSH chat server listening on port 2222
Clients connect with: ./main_client <server-ip>
Connected (internal): 127.0.0.1
Connected (internal): 127.0.0.1
Connected (internal): 127.0.0.1
Connected (internal): 127.0.0.1
^C
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
make clean && make
rm -f main_server main_client
gcc -o main_server main_server.c -lpthread -lssh
gcc -o main_client main_client.c -lpthread -lssh
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
./main_server
Internal chat listener on 127.0.0.1:8081
SSH chat server listening on port 2222
Clients connect with: ./main_client <server-ip>
Connected (internal): 127.0.0.1
Created room 1
Connected Clients:
[Room 1] Nick (127.0.0.1)
Connected (internal): 127.0.0.1
Joined room 1
Connected Clients:
[Room 1] Nick (127.0.0.1)
[Room 1] Sofia (127.0.0.1)
Connected Clients:
[Room 1] Nick (127.0.0.1)
Room 1 is now empty and closed.
Connected Clients:
No clients connected
^C
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
$

tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
$ ss -tnp | grep 2222
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
$ ss -tnp | grep 2222
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
$ ./main_client 127.0.0.1
Connecting to 127.0.0.1 via SSH tunnel...
SSH tunnel established.
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
$ ./main_client 127.0.0.1
Connecting to 127.0.0.1 via SSH tunnel...
SSH tunnel established.
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
$ ./main_client 127.0.0.1 new
Connecting to 127.0.0.1 via SSH tunnel...
SSH tunnel established.
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
$ ./main_client 127.0.0.1 new
Connecting to 127.0.0.1 via SSH tunnel...
SSH tunnel established.
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
$ ./main_client 127.0.0.1 new
Connecting to 127.0.0.1 via SSH tunnel...
SSH tunnel established.
Connected to 127.0.0.1 with new room number 1
Type your user name: Nick
Nick (SSH) joined the chat room!
Sofia (127.0.0.1) joined the chat room!
Sofia (127.0.0.1) left the room!
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
$
```

Figure 21. SSH Tunneling via libssh (Direct Port Forwarding)

A libssh implementation is used to reimplement the chat application with SSH tunneling by setting up Direct Port Forwarding from the remote SSH server binding to port 2222. It then forwards the communication to an internal chat daemon bound to 127.0.0.1:8081. Clients connect to the server using libssh and create a direct-tcpip forwarding channel to tunnel all chat communications to the server's internal listening port. The internal chat daemon is not accessible to clients. The command `ss -tnp | grep 2222` verifies that all clients are being bound at the SSH level to port 2222. The same output shows that the processes `main_server` and `main_client` are the SSH endpoint and SSH client respectively. On the server terminal, there are messages indicating the "SSH chat server listening on port 2222" and the "Internal chat listener on 127.0.0.1:8081". The SSH chat server listener and the chat server are set up to forward the clients' connections through an SSH tunnel, fulfilling the Direct Port Forwarding requirement.

Extra Credit 2:



```
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345 80x24
tcnj@tcnj-osc:~$ echo "hello this is a test" > test.txt
tcnj@tcnj-osc:~$ cd ~/Downloads/Proj4_CSC345/Proj4_CSC345/
tcnj@tcnj-osc:~/Downloads/Proj4_CSC345/Proj4_CSC345$ ^C
tcnj@tcnj-osc:~/Downloads/Proj4_CSC345/Proj4_CSC345$ echo "hello this is a test"
> test.txt
tcnj@tcnj-osc:~/Downloads/Proj4_CSC345/Proj4_CSC345$ cat received_test.txt
hello this is a test
tcnj@tcnj-osc:~/Downloads/Proj4_CSC345/Proj4_CSC345$ ss -tnp | grep 2222
ESTAB 0      0      127.0.0.1:2222      127.0.0.1:52918 users:(("main_serv
er",pid=1773522,fd=6))
ESTAB 0      0      127.0.0.1:52918      127.0.0.1:2222 users:(("main_clie
nt",pid=1773621,fd=3))
ESTAB 0      0      127.0.0.1:2222      127.0.0.1:40170 users:(("main_serv
er",pid=1773522,fd=7))
ESTAB 0      0      127.0.0.1:45336      127.0.0.1:2222 users:(("main_clie
nt",pid=1774073,fd=3))
ESTAB 0      0      127.0.0.1:2222      127.0.0.1:45336 users:(("main_serv
er",pid=1773522,fd=10))
ESTAB 0      0      127.0.0.1:40170      127.0.0.1:2222 users:(("main_clie
nt",pid=1773819,fd=3))
tcnj@tcnj-osc:~/Downloads/Proj4_CSC345/Proj4_CSC345$
```

Figure 22. File Transfer Verification & SSH Tunneling Verification

The commands that verify the file transfer in the previous terminal window are run in a shell outside of the chat client. The original file was created with the command `echo "hello this is a test" > test.txt`. `cat received_test.txt` confirms that the file has been received unmodified. The output of `ss -tnp | grep 2222` below that output shows that all the client connections to port 2222 from the chat tunnel remain open during as well as after the file transfer, which shows that the SSH chat tunnel was never interrupted.

```
tcnj@tcnj-osc: ~/Downloads/Pr... x tcnj@tcnj-osc: ~/Downloads/Proj4_... x tcnj@tcnj-osc: ~/Downloads/Proj... x tcnj@tcnj-osc: ~/Downloads/Proj... x
tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345/Proj4_CSC345$ ./main_server
Internal chat listener on 127.0.0.1: 8081
SSH chat server listening on port 22
Clients connect with: ./main_client
<server-ip>
Connected (internal): 127.0.0.1
Auto created room 1
Connected Clients:
[Room 1] Nick (127.0.0.1)
Connected (internal): 127.0.0.1
Joined room 1
Connected Clients:
[Room 1] Nick (127.0.0.1)
[Room 1] Sofia (127.0.0.1)
Connected (internal): 127.0.0.1
Joined room 1
Connected Clients:
[Room 1] Nick (127.0.0.1)
[Room 1] Sofia (127.0.0.1)
[Room 1] Das (127.0.0.1)

tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345$ ./main_client 127.0.0.1
connecting to 127.0.0.1 via SSH tunnel
SSH tunnel established.
server says the following options are available:
room 1: 1 person
choose the room number or type [new]
o create a new room: 1
connected to room 1
type your user name: Nick
Nick (SSH) joined the chat room!
[FT] To send a file: SEND <username> <filename>
i
ofia (127.0.0.1) joined the chat room!
Sofia (127.0.0.1)] hi
Nick (127.0.0.1)] echo "hello this is a test" > test.txt
as (127.0.0.1) joined the chat room!
[FT] Sofia wants to send you 'test.txt' (21 bytes). Accept? (Y/N): Y
[FT] Received 'test.txt' -> saved as 'received_test.txt' (21 bytes).
FILE_DONE Transfer of 'test.txt' complete (21/21 bytes).

tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345$ ./main_client 127.0.0.1
connecting to 127.0.0.1 via SSH tunnel
SSH tunnel established.
server says the following options are available:
room 1: 1 person
choose the room number or type [new]
o create a new room: 1
connected to room 1
type your user name: Sofia
Sofia (SSH) joined the chat room!
[FT] To send a file: SEND <username> <filename>
i
Nick (127.0.0.1)] hi
Nick (127.0.0.1)] echo "hello this is a test" > test.txt
as (127.0.0.1) joined the chat room!
END Nick test.txt
FILE_WAIT Waiting for receiver to accept...
[FT] Sent 21 bytes.
FILE_DONE Transfer of 'test.txt' complete (21/21 bytes).

tcnj@tcnj-osc: ~/Downloads/Proj4_CSC345$ ./main_client 127.0.0.1
connecting to 127.0.0.1 via SSH tunnel
SSH tunnel established.
server says the following options are available:
room 1: 2 people
choose the room number or type [new]
o create a new room: 1
connected to room 1
type your user name: Das
Das (SSH) joined the chat room!
[FT] To send a file: SEND <username> <filename>
```

Figure 23. File Transfer Between Clients

The four terminal windows show the entire file transfer process between the three clients served by Room 1 (Nick 2nd from left, Sofia and Das), communicated over an SSH tunnel. In the window belonging to Nick, you can see the SEND Sofia *test.txt* command, and the FILE_WAIT message while waiting for acceptance. Sofia's terminal (third from left) shows the FILE_OFFER with sender, filename and size (21 bytes), her Y acceptance, the FILE_DONE Transfer of 'test.txt' complete (21/21 bytes) and the [FT] Received 'test.txt' -> saved as 'received_test.txt'. Das's terminal (far right) shows that FILE_* messages are not visible despite him being in the room, showing room isolation.